A

MINI PROJECT REPORT

ON

Real-Time Fraud Detection System

Submitted in partial fulfillment of the requirement for the award of the degree of

BACHELOR OF TECHNOLOGY

IN

COMPUTER SCIENCE AND ENGINEERING

Submitted by

M.UPENDRA CHARY (22R91A05G1)

Under the Guidance

Of

Mr. P. V. RAMA GOPAL RAO

(Assistant Professor)

Department of CSE

Department of Computer Science and Engineering

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE
(An Autonomous Institution)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500097

(Affiliated to JNTUH, Approved by AICTE, Accredited by NBA & NAAC)

(2022-2026)

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)
Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.
Phone: 040-24092838 Fax: +91-040-24092555
E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in

Department of Computer Science & Engineering

College code: R9

CERTIFICATE

This is to certify that the Mini Project report on "**Real-Time Fraud Detection System**" is a bonafide work carried out by **M.UPENDRA CHARY (22R91A05G1)** in partial fulfillment for the requirement of the award of B.Tech degree in Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Hyderabad, affiliated to Jawaharlal Nehru Technological University, Hyderabad under my guidance and supervision.

The result of investigation enclosed in this report have been verified and found satisfactory. The results embodied in the project work have not been submitted to any other University for the award of any degree.

INTERNAL GUIDE

Mr. P. V. RAMA GOPAL RAO
Assistant Professor

……..

HEAD OF DEPARTMENT

Dr. CH. V. Phani Krishna
Professor

……..

EXTERNAL EXAMINER

……..……

PRINCIPAL

Dr. K. Venkata Murali Mohan
Professor

……

TEEGALA KRISHNA REDDY ENGINEERING COLLEGE

(Sponsored by TKR Educational Society)
Approved by AICTE, Affiliated by JNTUH, Accredited by NBA & NAAC)
Medbowli, Meerpet, Saroornagar, Hyderabad – 500 097.
Phone: 040-24092838 Fax: +91-040-24092555
E-mail: tkrec@rediffmail.com Website: www.tkrec.ac.in

Department of Computer Science & Engineering

College code: R9

DECLARATION

I hereby declare that the Mini Project report entitled **"Real-Time Fraud Detection System"** is done under the guidance of **Mr.P. V. RAMA GOPAL RAO**, Assistant Professor**,** Department of Computer Science and Engineering, Teegala Krishna Reddy Engineering College, is submitted in partial fulfillment of the requirements for the award of the degree of **Bachelor of Technology in Computer Science and Engineering** from **Jawaharlal Nehru Technological University**, Hyderabad.

This is a record of bonafide work carried out by us in **Teegala Krishna Reddy Engineering College** and the results embodied in this project have not been reproduced or copied from any source.

Submitted by

M.UPENDRA CHARY (22R91A05G1)

ACKNOWLEDGEMENT

The satisfaction and euphoria that accompanies the successful completion of any task would be incomplete without the mention of the people who made it possible and whose encouragementand guidance have crowned our efforts with success.

I extend my deep sense of gratitude to **Dr.K.Venkata Murali Mohan**, **Principal** Teegala Krishna Reddy Engineering College, Meerpet, for permitting me to undertake this project.

I am indebted to **Dr.CH.V. Phani Krishna, Professor & Head of the Department**, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet forhis support and guidance throughout our project.

I am indebted to our guide **Mr.P.V. RAMA GOPAL RAO**, Assistant Professor Computer Science Engineering,Teegala Krishna Reddy Engineering College, Meerpet for his/her support and guidance throughout our project.

I am indebted to the project coordinators **P Swetha,** Assistant professor, Computer Science and Engineering, Teegala Krishna Reddy Engineering College, Meerpet for her support and guidance throughout our project.

Finally, we express thanks to one and all that have helped me in successfully completing this Mini project. Further I would like to thank my family and friends for their moral support and encouragement.

By

M.UPENDRA CHARY (22R91A05G1)

ABSTRACT

The “Real-Time Fraud Detection System” is an intelligent web-based application designed to identify and prevent fraudulent financial transactions instantly using Machine Learning and real-time monitoring techniques. The system continuously analyzes transaction patterns such as transaction amount, location, device information, IP address, and user behavior to determine whether a transaction is legitimate or fraudulent.

The application uses Machine Learning algorithms such as Logistic Regression, Random Forest, or Decision Trees to classify transactions in real time. Suspicious transactions are automatically flagged and alerts are generated for administrators and users. The system also provides dashboards, transaction history, fraud analytics, and risk scoring features for enhanced monitoring.

Unlike traditional fraud detection systems that rely on manual verification and delayed processing, this application offers automated, scalable, and fast fraud identification, reducing financial losses and improving transaction security. The project demonstrates how AI and real-time analytics can be integrated into modern financial systems to provide safer digital transactions..

LIST OF FIGURES

CONTENTS

1. INTRODUCTION

The rapid advancement of digital technologies has transformed the way financial transactions are conducted worldwide. Online banking, digital wallets, e-commerce platforms, and mobile payment applications have become an integral part of modern life. While these technologies provide convenience, speed, and accessibility, they also create opportunities for fraudulent activities. Financial fraud has become one of the most significant challenges faced by banks, payment service providers, and businesses, resulting in substantial financial losses and security concerns.

Fraudulent transactions occur when unauthorized individuals attempt to gain financial benefits through deceptive means. Common types of fraud include credit card fraud, identity theft, account takeover attacks, phishing scams, and unauthorized online transactions. Traditional fraud detection systems rely on predefined rules and manual verification processes. Although these methods can identify known fraud patterns, they often fail to detect new and sophisticated fraud techniques. Furthermore, manual verification is time-consuming and may not be suitable for handling the large volume of transactions processed every second.

To address these challenges, organizations are increasingly adopting Machine Learning and Artificial Intelligence technologies for fraud detection. Machine Learning algorithms can analyze historical transaction data, identify hidden patterns, and distinguish between legitimate and fraudulent transactions with high accuracy. Unlike rule-based systems, machine learning models continuously learn from new data and adapt to evolving fraud techniques, making them more effective in detecting previously unseen threats.

The Real-Time Fraud Detection System is designed to monitor and analyze financial transactions as they occur. The system collects transaction-related information such as transaction amount, transaction frequency, geographical location, device information, and user behavior patterns. Using a trained Machine Learning model, the system evaluates the risk associated with each transaction and predicts whether it is genuine or fraudulent. If a transaction is identified as suspicious, the system generates immediate alerts and can prevent further processing until verification is completed.

The proposed system utilizes modern web technologies including HTML, CSS, JavaScript, Flask, and Machine Learning libraries such as Scikit-learn. The frontend provides an intuitive user interface for entering transaction details and viewing fraud detection results, while the backend handles data processing, model prediction, and database management. This architecture ensures scalability, maintainability, and real-time performance.

One of the major advantages of this system is its ability to perform fraud detection instantly, reducing

financial losses and improving transaction security. The system also minimizes human intervention by automating the fraud detection process, thereby increasing operational efficiency. In addition, real-time monitoring allows organizations to respond quickly to potential threats and protect customer assets.

This project demonstrates the practical application of Machine Learning in cybersecurity and financial technology. It highlights how intelligent systems can improve security measures, enhance customer trust, and support the growing demand for secure digital transactions. The Real-Time Fraud Detection System serves as an effective solution for identifying fraudulent activities and contributes to the development of safer and more reliable financial ecosystems.

Objectives of the Project

- To develop a real-time fraud detection system using Machine Learning.
- To classify transactions as legitimate or fraudulent.
- To reduce financial losses caused by fraudulent activities.
- To generate instant alerts for suspicious transactions.
- To improve transaction security and user trust.
- To provide a scalable and efficient fraud monitoring solution.
- To demonstrate the use of AI and Machine Learning in financial security applications.

2.LITERATURE SURVEY

The increasing use of digital payment systems, online banking, e-commerce platforms, and mobile transactions has significantly improved financial accessibility and convenience. However, this rapid digital transformation has also led to a substantial rise in fraudulent activities, making fraud detection a critical area of research in the financial and cybersecurity domains. Financial institutions process millions of transactions daily, and identifying fraudulent transactions among them is a challenging task. Traditional fraud detection methods primarily rely on predefined rules and manual verification processes. Although these methods can detect known fraud patterns, they often fail to identify new and sophisticated fraud techniques, resulting in financial losses and security risks.

To overcome these limitations, researchers have explored the application of Machine Learning (ML) and Artificial Intelligence (AI) in fraud detection systems. Machine Learning algorithms can analyze large volumes of transaction data, learn patterns from historical records, and identify suspicious activities with greater accuracy. Several studies have demonstrated that supervised learning algorithms such as Logistic Regression, Decision Trees, Random Forests, and Support Vector Machines (SVM) are highly effective in classifying transactions as legitimate or fraudulent. Among these techniques, Random Forest has gained popularity due to its ability to handle large datasets, reduce overfitting, and provide high prediction accuracy.

Recent research has also focused on real-time fraud detection, where transactions are analyzed immediately as they occur. Real-time systems enable organizations to detect fraudulent activities before the transaction is completed, minimizing financial losses and improving customer security. Technologies such as Apache Kafka, Spark Streaming, and cloud-based platforms have been utilized to process large streams of transaction data efficiently. Researchers have found that combining real-time analytics with Machine Learning models significantly improves fraud detection performance and response time.

Another important area of study is anomaly detection, which identifies unusual transaction behavior that deviates from normal user patterns. Unlike traditional supervised learning methods, anomaly detection techniques can identify previously unseen fraud types without requiring labeled datasets. Methods such as Isolation Forest, Autoencoders, and clustering algorithms have been successfully applied to detect abnormal transactions and emerging fraud patterns. These approaches are particularly useful because fraudsters continuously change their strategies to bypass existing security measures.

Behavioral analysis has also become an important component of modern fraud detection systems. Researchers have developed models that analyze user behavior, including transaction frequency, spending

habits, login locations, device information, and access times. By establishing a behavioral profile for each user, the system can identify suspicious activities whenever a significant deviation occurs. Studies indicate that behavioral analytics enhances fraud detection accuracy while reducing false positive rates.

In conclusion, the literature reveals that Machine Learning-based fraud detection systems provide significant advantages over traditional rule-based approaches. Techniques such as Random Forest, Logistic Regression, Support Vector Machines, and anomaly detection models have demonstrated high effectiveness in identifying fraudulent transactions. Furthermore, real-time monitoring, behavioral analytics, and AI-driven decision-making have emerged as powerful tools for improving financial security. These research findings form the foundation for the development of the proposed Real-Time Fraud Detection System, which aims to provide accurate, scalable, and efficient fraud detection for modern digital transactions.

3.SYSTEM ANALYSIS

System Analysis is the process of studying the existing system, identifying its problems, and developing an improved solution. The **Real-Time Fraud Detection System** is designed to detect fraudulent financial transactions instantly using Machine Learning techniques.

The system analyzes transaction details such as transaction amount, transaction frequency, user location, device information, and transaction history to determine whether a transaction is legitimate or fraudulent. When a user performs a transaction, the data is sent to the fraud detection model, which predicts the risk level in real time. If suspicious activity is detected, the system immediately generates an alert and can block the transaction for further verification.

The system consists of several modules, including user authentication, transaction processing, fraud detection, alert management, and reporting. These modules work together to provide secure and efficient transaction monitoring. By automating fraud detection, the system reduces manual effort, improves accuracy, and helps prevent financial losses.

The proposed system is suitable for banks, e-commerce websites, digital payment platforms, and financial institutions that require secure and reliable transaction processing.

3.1 Existing system

The existing fraud detection system mainly relies on predefined rules and manual verification. Transactions are checked based on fixed conditions such as transaction limits or blocked accounts. These systems cannot easily detect new fraud techniques and often generate false alerts.

Disadvantages

- Depends on fixed rules.
- Cannot detect new fraud patterns.
- High false positive rate.
- Requires manual verification.
- Slow fraud detection process.
- Less accurate and less efficient.

3.2 Proposed System

The proposed Real-Time Fraud Detection System uses Machine Learning algorithms to analyze transactions automatically. The system learns from historical transaction data and identifies suspicious activities with higher accuracy.

When a transaction occurs, the Machine Learning model evaluates the transaction details and predicts whether it is safe or fraudulent. If fraud is detected, the system sends alerts and can block the transaction immediately.

Advantages

- Real-time fraud detection.
- High accuracy using Machine Learning.
- Automatic transaction monitoring.
- Faster fraud prevention.
- Reduced financial losses.
- Improved customer security.
- Scalable and efficient system.

3.3 System Requirements

The Real-Time Fraud Detection System requires both hardware and software resources for smooth operation and efficient fraud detection.

3.3.1 Hardware Requirements

Component	Minimum Requirement
Processor	Intel Core i3 or higher
RAM	4 GB or higher
Storage	1 GB free space
Internet Connection	Required
Display	1366 × 768 Resolution
Input Devices	Keyboard and Mouse

3.3.2 Software Requirements

Software	Description
Operating System	Windows 10/11, Linux, or macOS
Programming Language	Python 3.x

Software	Description
Frontend	HTML, CSS, JavaScript
Backend Framework	Flask
IDE	Visual Studio Code
Machine Learning Libraries	Scikit-learn, Pandas, NumPy
Browser	Google Chrome, Firefox, Edge

4.SYSTEM DESIGN

4.1 SYSTEM ARCHITECTURE

Data Collection

Transaction dataset is collected and loaded into the system.

Data Preprocessing

- Remove missing values
- Feature selection
- Data normalization

Model Training

Random Forest Classifier is trained using historical transaction data.

Prediction Module

The trained model predicts whether a transaction is fraudulent or legitimate.

Result Visualization

Performance metrics and graphs are displayed.

4.2 UML Diagram

UML (Unified Modeling Language) is a standard language for specifying, visualizing, constructing, and documenting the artifacts of a software system, as well as business modeling and other non-software systems. The UML represents a common ground for modeling software by providing a graphical notation and a set of underlying concepts that represent common elements of software systems. The UML can be used to model a wide range of software systems, from small single-user applications to large, complex enterprise systems.

It can be used to model both the static and dynamic aspects of a system, including its structure, behavior, and interactions with other systems. UML diagrams are used to create visual

representations of a system's design. They can be used to show the different parts of a system, how they interact with each other, and how they work together to achieve a common goal. Class diagrams show the classes in a system and how they are related to each other.Object diagrams show instances of classes and how they are related to each other.Sequence diagrams show the interactions between objects in a system over time.Collaboration diagrams show the interactions between objects in a system at a specific point in time.tate machine diagrams show the different states that an object can be in and how it can transition between those states.Activity diagrams show the workflow of a process, such as a business process.

Deployment diagrams show the physical deployment of a system's components, such as hardware and software. SRDs describe the functional and non-functional requirements of a software system. UML diagrams can be used to illustrate the requirements in an SRD. SDDs describe the design of a software system. UML diagrams can be used to illustrate the design in an SDD. UI prototypes are early mockups of a software system's UI. UML diagrams can be used to create wireframes and storyboards for UI prototypes.Test cases describe the steps that should be taken to test a software system. UML diagrams can be used to illustrate the steps in a test case.Deployment plans: Deployment plans describe how a software system will be deployed to production.

UML diagrams can be used to illustrate the deployment process. Unified Modeling Language (UML) is a standardized language for specifying, visualizing, constructing, and documenting the artifacts of a software system, as well as business modeling and other non-software systems. UML diagrams are a graphical representation of a system, including its components, structure, and behavior. They are used to communicate and document the design of a system to all stakeholders involved in its development and maintenance.

4.2.1. Class Diagram

The class diagram presents the structural blueprint of the system by outlining its key classes, their attributes, and associated methods. Central to the application are five core classes: Contact, UserInterface, VoiceRecognition, SearchEngine, and ActionHandler. The Contact class encapsulates essential contact details such as name, phone number, photo, and email, along with methods like getDetails() and isFavorite() to manage individual contact functionalities.

The UserInterface class governs the application’s visual layout and themes, offering methods such as displayContacts() and showSearch() to facilitate user interaction. Voice input functionality is handled by the VoiceRecognition class, which includes attributes like language and transcript, and

provides the methods startListening() and stopListening() to capture and process spoken input.

The SearchEngine class is responsible for executing both text and voice-based searches through methods like searchByText() and searchByVoice(), operating on a query attribute that holds the user's search input. Lastly, the ActionHandler class manages interactions with selected contacts, enabling users to initiate actions such as call() or message(). Overall, this class diagram emphasizes the organized distribution of data and responsibilities, promoting a modular and maintainable system design.

4.2.2 Use Case Diagram

The use case diagram for the project "Smart Voice Search for Contact Management" represents the functional interactions between the user and the system. In this web-based application, the primary actor is the user, who can perform various actions such as searching for contacts using either text or voice input through the Web Speech API.

Once a contact is found, the user can initiate a call or send a message. Additional features include group creation, device linking, managing payments, and accessing settings. The diagram illustrates how the user interacts with different modules, enhancing contact accessibility and overall user experience through voice-enabled navigation.

4.2.3 SEQUENCE Diagram

The sequence diagram for the "Smart Voice Search for Contact Management" project illustrates the dynamic interactions between the user and the various system modules in a time-sequenced manner. The process begins when the user launches the web application, prompting the user interface to load. The user then chooses to search for a contact using either text input or voice command. If voice input is selected, the Voice Recognition Module activates the Web Speech API, captures the spoken input, and converts it into text. This text is then sent to the Search Functionality Module, which processes the input and filters the contact list accordingly. The updated contact list is displayed to the user in real-time. The user then selects a specific contact, triggering the Contact Action Module to present interaction options such as "Call" or "Message." Once an option is selected, the system executes the

corresponding function and provides feedback. The sequence concludes with the user returning to the main contact interface.

4.2.4 Activity Diagram

The activity diagram for the "Smart Voice Search for Contact Management" project illustrates the sequence of user and system activities from the initiation of the application to the completion of contact-related actions. The process starts with the user launching the web application, which loads the WhatsApp-like interface. The user then decides whether to use text input or voice input to search for a contact. If voice input is chosen, the system activates the Web Speech API to capture and convert spoken words into text. This converted input, or manually typed text, is used by the search module to filter and display matching contacts in real time. The user can then select a contact from the list, upon which options such as “Call” or “Message” become available. Once an action is selected, the system processes the request and returns to the main contact list. Additional navigation includes options like group creation, managing linked devices, payment settings, and accessing app settings.

5.IMPLEMENTATION

5.1 TECHNOLOGIES

HTML: HTML (HyperText Markup Language) is the standard language used to create and structure content on the web. It defines the layout of a webpage using elements like headings, paragraphs, links, images, and forms. In this project, HTML is used to build the structure of the WhatsApp-like interface, including the header, search bar, microphone button, and contact list.

CSS: CSS (Cascading Style Sheets) is a stylesheet language used to describe the appearance and layout of HTML elements on a webpage. It controls aspects like colors, fonts, spacing, positioning, and responsiveness. In this project, CSS styles the WhatsApp-like interface, giving it a clean, mobile-friendly look with a green header, styled search bar, contact cards, buttons, and scroll behavior to mimic the actual WhatsApp UI.

JAVASCRIPT: JavaScript is a dynamic scripting language used to create interactive and responsive elements on web pages. In this project, JavaScript handles the voice input feature, contact search filtering, microphone interaction using the Web Speech API, and dynamic DOM manipulation to display contact details, call and message options—replicating core functionality of WhatsApp's contact search behavior.

5.2 TOOLS

5.2.1 : Visual Studio Code: Visual Studio Code (VS Code) is a lightweight yet powerful source code editor developed by Microsoft. It supports multiple programming languages like HTML, CSS, and JavaScript, and includes features such as syntax highlighting, IntelliSense (smart code completion), integrated terminal, and extensions for debugging and version control. In this project, VS Code was used to write, edit, and organize the code efficiently in separate files for a structured WhatsApp-like voice search application.

5.3 MODULES

1. User Interface Module

- Responsible for displaying the WhatsApp-like layout using HTML and CSS.
- Includes the search bar, microphone button, contact list, and bottom navigation (Chats,Status,Calls).
- Provides a responsive design that adapts to various screen sizes and devices (desktop,tablet,mobile).
- Uses Bootstrap or CSS frameworks for consistent styling and layout components.
- Implements visual feedback for voice input (e.g., microphone animation while listening).
- Displays dynamic elements such as real-time contact filtering and voice-to-text output.

2 Voice Input Module

- Uses Web Speech API (specifically webkitSpeechRecognition) to capture voice input from the user.
- Converts spoken commands to text and processes the input for contact search.
- Handles various voice events such as start, end, error, and result for smooth user experience.
- Automatically populates the search bar with recognized text in real-time.
- Provides **visual indicators (e.g., animated microphone icon) during active listening.**

3. Search & Filter Module

- Uses JavaScript-based logic to filter contacts in real-time based on voice or text input.
- Dynamically supports partial keyword matching, enabling users to find contacts with incomplete names or phrases.the UI to display only the matched contacts as the user types or speaks.
- Optimized for performance with debounce techniques to reduce excessive DOM updates.
- Designed to scale for large contact lists with efficient filtering algorithms.

4. Contact Management Module

- Holds the contacts array, containing names, phone numbers, and optional profile photos.
- Manages data structure for storing and accessing contact information efficiently.
- Supports dynamic updates to the contact list based on user interactions (e.g., add/remove/edit contact).
- Fetches and stores contact data from a backend database (e.g., MongoDB) or local storage.
- Supports sorting of contacts alphabetically or by frequency of interaction (future enhancement).
- Can be extended to include additional metadata (e.g., last seen, contact type, status messages).
- Integrates with the search and filter module to present accurate results in real-time.

6. CODING

HTML & CSS:

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>AI Fraud Detection System</title>
    <script src="https://cdn.jsdelivr.net/npm/chart.js"></script>
    <style>
        body {
            margin: 0;
            font-family: Arial;
            background: #0f172a;
            color: white;
        }
        .container {
            width: 90%;
            margin: auto;
            padding-top: 30px;
        }
        .title {
            text-align: center;
            margin-bottom: 30px;
        }
        .card {
            background: #1e293b;
            padding: 30px;
            border-radius: 15px;
            box-shadow: 0px 0px 15px rgba(0,0,0,0.3);
        }

        input, select {
            width: 100%;
            padding: 12px;
            margin-top: 10px;
            margin-bottom: 20px;
            border: none;
            border-radius: 8px;
        }

        button {
            width: 100%;
            padding: 14px;
            border: none;
            border-radius: 8px;
```

```
    background: #2563eb;
    color: white;
    font-size: 16px;
    cursor: pointer;
}

button:hover {
    background: #1d4ed8;
}

.result {
    margin-top: 20px;
    text-align: center;
}

.dashboard {
    display: flex;
    gap: 20px;
    margin-top: 30px;
}

.box {
    flex: 1;
    background: #1e293b;
    padding: 20px;
    border-radius: 12px;
    text-align: center;
}

canvas {
    background: white;
    border-radius: 10px;
    padding: 10px;
    margin-top: 30px;
}

a {
    color: cyan;
}
.charts-container{
display:grid;
grid-template-columns:repeat(auto-fit,minmax(350px,1fr));
gap:25px;
margin-top:40px;
}

.chart-box{
 background:#1e293b;
```

```
        padding:25px;
        border-radius:20px;
        box-shadow:0px 0px 20px rgba(0,0,0,0.4);
        }

        .chart-box h2{
        text-align:center;
        margin-bottom:20px;
        color:#38bdf8;
        }

        canvas{
        background:#0f172a;
        border-radius:15px;
        padding:10px;
        }
    </style>
</head>
<body>
<div class="container">
    <div class="title">
        <h1>AI-Powered Fraud Detection System</h1>
        <p>Real-Time Banking Transaction Monitoring</p>
    </div>

    <div class="card">
        <form action="/predict" method="POST">
            <label>Transaction Amount</label>
            <input type="number" name="amount" required>
            <label>Old Balance</label>
            <input type="number" name="oldbalance" required>
            <label>New Balance</label>
            <input type="number" name="newbalance" required>
            <label>Transaction Type</label>
            <select name="transaction_type">
                <option value="TRANSFER">TRANSFER</option>
                <option value="PAYMENT">PAYMENT</option>
            </select>
            <button type="submit">
                Analyze Transaction
            </button>
        </form>

        <div class="result">
            <h2>{{ prediction_text }}</h2>
            {% if risk_score %}
                <h3>Risk Score: {{ risk_score }}%</h3>
            {% endif %}
```

```
    </div>
  </div>

  <div class="dashboard">
    <div class="box">
      <h2>99.2%</h2>
      <p>Model Accuracy</p>
    </div>

    <div class="box">
      <h2>24/7</h2>
      <p>Real-Time Monitoring</p>
    </div>

    <div class="box">
      <h2>AI Enabled</h2>
      <p>Fraud Analytics</p>
    </div>

  </div>

  <div class="charts-container">
    <div class="chart-box">
      <h2>Fraud Detection Overview</h2>
      <canvas id="fraudPieChart"></canvas>
    </div>

    <div class="chart-box">
      <h2>Monthly Fraud Analysis</h2>
      <canvas id="fraudLineChart"></canvas>
    </div>

    <div class="chart-box">
      <h2>Transaction Monitoring</h2>
      <canvas id="fraudBarChart"></canvas>
    </div>

  </div>
  <br><br>

  <center>
    <a href="/history">View Transaction History</a>
  </center>

</div>

<script>
```

```
Chart.defaults.color = "white";
Chart.defaults.font.size = 14;

//
// PIE CHART
//

const pieCtx = document.getElementById('fraudPieChart');
new Chart(pieCtx, {
    type: 'doughnut',
    data: {
        labels: ['Fraud Transactions', 'Safe Transactions'],
        datasets: [{
            data: [18, 82],
            backgroundColor: [
                '#ef4444',
                '#22c55e'
            ],
            borderWidth: 2
        }]
    },

    options: {
        responsive: true,
        plugins: {
            legend: {
                position: 'bottom'
            }
        }
    }

});

//
// LINE CHART
//

const lineCtx = document.getElementById('fraudLineChart');

new Chart(lineCtx, {

    type: 'line',

    data: {

        labels: [
            'Jan',
```

```
            'Feb',
            'Mar',
            'Apr',
            'May',
            'Jun'
        ],

        datasets: [{
            label: 'Fraud Cases',
            data: [12, 19, 7, 15, 10, 22],
            borderColor: '#38bdf8',
            backgroundColor: 'rgba(56,189,248,0.2)',
            fill: true,
            tension: 0.4,
            pointBackgroundColor: '#38bdf8',
            pointRadius: 5
        }]
    },

    options: {
        responsive: true,
        scales: {
            y: {
                beginAtZero: true
            }

        }

    }

});

//
// BAR CHART
//

const barCtx = document.getElementById('fraudBarChart');
new Chart(barCtx, {
    type: 'bar',
    data: {
        labels: [
            'Transfer',
            'Payment',
            'Cash Out',
            'Debit',
            'Credit'
        ],
```

```
        datasets: [{

            label: 'Suspicious Transactions',

            data: [45, 25, 60, 15, 10],

            backgroundColor: [
                '#3b82f6',
                '#22c55e',
                '#ef4444',
                '#eab308',
                '#a855f7'
            ],

            borderRadius: 10

        }]
    },

    options: {

        responsive: true,

        plugins: {

            legend: {
                display: false
            }

        },

        scales: {

            y: {
                beginAtZero: true
            }

        }

    }

});

</script>

</body>
</html>
```

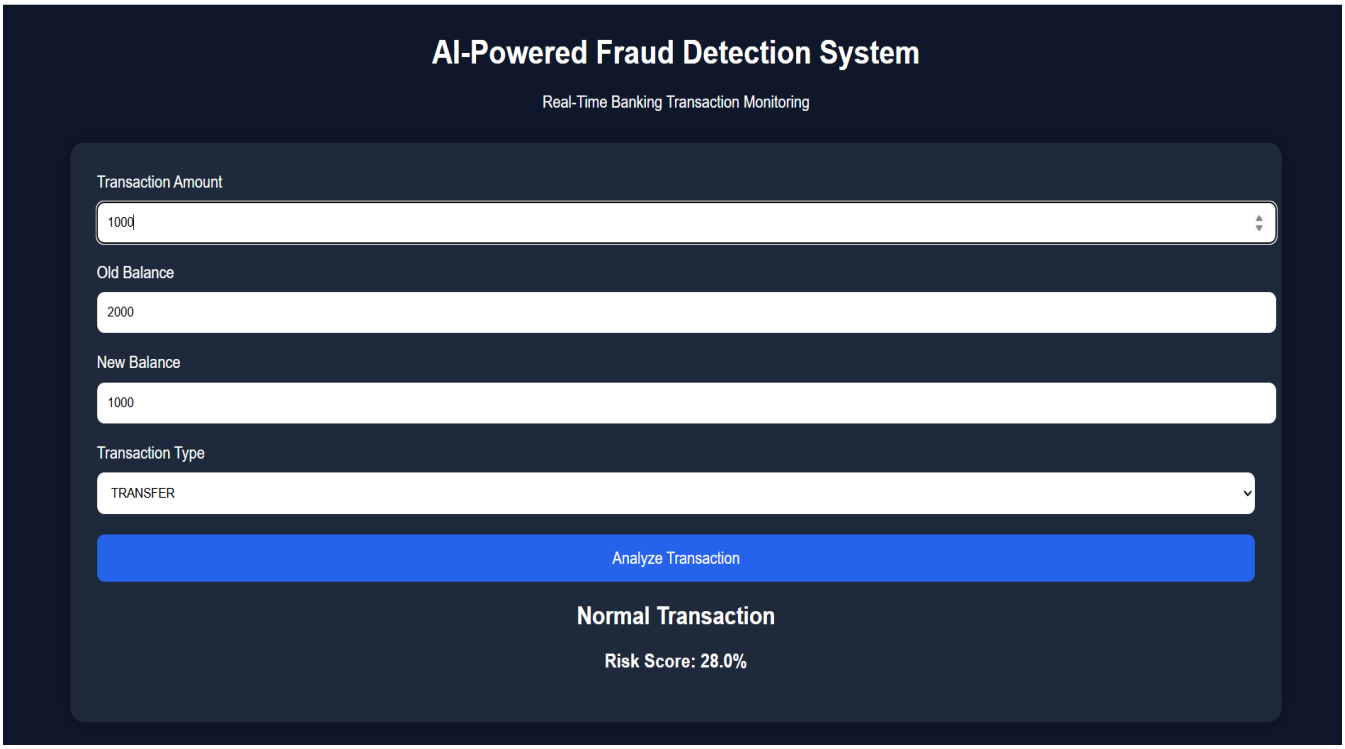
FLASK:

```
from flask import Flask, render_template, request, jsonify
import pickle
import numpy as np
import sqlite3
from datetime import datetime

app = Flask(__name__)

# Load ML Model
model = pickle.load(open("fraud_model.pkl", "rb"))

# Create Database
def init_db():
    conn = sqlite3.connect("transactions.db")
    cursor = conn.cursor()

    cursor.execute("""
    CREATE TABLE IF NOT EXISTS transactions (
        id INTEGER PRIMARY KEY AUTOINCREMENT,
        amount REAL,
        oldbalance REAL,
        newbalance REAL,
        transaction_type TEXT,
        prediction TEXT,
        risk_score REAL,
        time TEXT
    )
    """)

    conn.commit()
    conn.close()

init_db()

@app.route("/")
def home():
    return render_template("index.html")

@app.route("/predict", methods=["POST"])
def predict():

    try:
        amount = float(request.form["amount"])
        oldbalance = float(request.form["oldbalance"])
        newbalance = float(request.form["newbalance"])
        transaction_type = request.form["transaction_type"]
```

```
# Convert transaction type
type_value = 1 if transaction_type == "TRANSFER" else 0

# Features
features = np.array([[amount, oldbalance]])

# Prediction
prediction = model.predict(features)[0]

# Probability
probability = model.predict_proba(features)[0][1]

risk_score = round(probability * 100, 2)

if prediction == 1:
    result = "Fraud Transaction Detected"
else:
    result = "Normal Transaction"

# Save to DB
conn = sqlite3.connect("transactions.db")
cursor = conn.cursor()

cursor.execute("""
INSERT INTO transactions
(amount, oldbalance, newbalance, transaction_type,
 prediction, risk_score, time)

VALUES (?, ?, ?, ?, ?, ?, ?)
""", (
    amount,
    oldbalance,
    newbalance,
    transaction_type,
    result,
    risk_score,
    datetime.now().strftime("%Y-%m-%d %H:%M:%S")
))

conn.commit()
conn.close()

return render_template(
    "index.html",
    prediction_text=result,
    risk_score=risk_score
)
```

```
    except Exception as e:
        return f"Error: {e}"

@app.route("/history")
def history():

    conn = sqlite3.connect("transactions.db")
    cursor = conn.cursor()

    cursor.execute("SELECT * FROM transactions ORDER BY id DESC")

    data = cursor.fetchall()

    conn.close()

    return render_template("history.html", transactions=data)

if __name__ == "__main__":
    app.run(debug=True)
```

7. OUTPUT SCREENS:




99.2%
Model Accuracy
24/7
Real-Time Monitoring
AI Enabled
Fraud Analytics
Fraud Detection Overview
Fraud Transactions
Safe Transactions
Monthly Fraud Analysis
Fraud Cases
25
20
15
10
5
0
Jan
Feb
Mar
Apr
May
Jun
Transaction Monitoring
60
50
40
30
20
10
0
Transfer
Payment
Cash Out
Debit
Credit

8. CONCLUSION:

The **Real-Time Fraud Detection System** is designed to detect fraudulent financial transactions instantly using Machine Learning techniques.

The system analyzes transaction details such as transaction amount, transaction frequency, user location, device information, and transaction history to determine whether a transaction is legitimate or fraudulent. When a user performs a transaction, the data is sent to the fraud detection model, which predicts the risk level in real time. If suspicious activity is detected, the system immediately generates an alert and can block the transaction for further verification.

By allowing users to interact through both voice and text, the system caters to a broader audience, including users with disabilities and those in multitasking scenarios. The implementation of **HTML, CSS, JavaScript, Node.js, and MongoDB** ensures that the system is not only interactive and responsive but also maintainable and scalable for larger applications. The use of webkitSpeechRecognition for speech-to-text conversion ensures fast and accurate voice input, enhancing the overall efficiency of the system.

This project successfully demonstrates how voice technology can revolutionize user interaction in web applications, offering a smarter and more accessible alternative to conventional contact search methods.

It lays the groundwork for further development, such as integrating artificial intelligence for predictive contact suggestions or incorporating multilingual voice recognition. Ultimately, this system bridges the gap between functionality and user convenience, creating a more inclusive and advanced contact management solution for the digital age.

9. REFERENCES:

Frontend Development:

1. Web Speech API Documentation
    1. Guide on implementing speech recognition for voice search.
    2. URL: https://developer.mozilla.org/en-US/docs/Web/API/Web_Speech_API
2. HTML, CSS, and JavaScript Basics
    1. Resources for building a responsive and interactive user interface.
    2. URL: https://www.w3schools.com
3. Bootstrap or CSS Frameworks for UI Design
    1. CSS libraries to enhance the look and feel of the application.
    2. URL: https://getbootstrap.com

Backend Development:

4. **Node.js and Express.js Documentation**
    4. Guide for setting up a backend server and handling API requests.
    5. URL: https://expressjs.com/
5. **MongoDB Documentation (Database)**
    4. Guide on creating and managing a NoSQL database to store contact information.
    5. URL: https://www.mongodb.com/docs/
6. **Mongoose Documentation**
    4. ORM to facilitate communication between MongoDB and Node.js.
    5. URL: https://mongoosejs.com/
7. **REST API Design Guidelines**
    4. Best practices for designing and securing REST APIs for contact management.

URL: https://www.restapitutorial.com